

Fundamentals of program development

Data validation, control flow,
exceptions and generators

Agenda

Data validation

Control structures

Useful idioms for loops and conditions

Searching in Lists

Generators

Exceptions

Data Validation

Error 1 in date input: unacceptable values for eval()

```
def inputDate(prompt):  
    year,month,day = eval(input(prompt))  
    return year,month,day
```

We had this easy but naive function for input of dates in **datediff.py** in Lecture 2.

A lot can go wrong when it takes input from the user:

```
Enter start date, in format year,month,date: 1970,01,01
```

```
Traceback (most recent call last):
```

```
File "datediff.py", File "<string>", line 1  
1970,01,01  
    ^
```

```
SyntaxError: invalid token
```

```
>>> 01  
File "<stdin>", line 1  
01  
    ^
```

```
SyntaxError: invalid token
```

01 for instance is not an acceptable int-literal.

It makes sense for usual numbers but not for dates. How would the user know that?

Error 2: vulnerability to attacks

```
Enter start date...: print(100*'haha I am an attacker!\n')
```

```
haha I am an attacker!
```

```
haha I am an attacker!
```

```
haha I am an attacker!
```

```
haha I am an attacker!
```

```
haha I am an attacker!
```

```
haha I am an attacker!
```

```
haha I am an attacker!
```

```
Traceback (most recent call last):
```

```
File "datediff.py", line 61, in main
```

```
    year1,month1,day1 = inputDate("start date")
```

```
File "datediff.py", line 57, in inputDate
```

```
    year,month,day = eval(input("Enter " + date + " in format  
year,month,day: "))
```

```
TypeError: cannot unpack non-iterable NoneType object
```

eval() is also problematic for security reasons.

Here the attacker has tested if we have naively used **eval()**. What would be his next step?

In addition there is also this error message which reveals information about the internals of the program. This helps the attacker too!!!

Version 2: get rid of eval

```
def inputDate(prompt):  
    year,month,day = input(prompt).split(',')  
    return int(year),int(month),int(day)
```

```
>>> int("02")  
2
```

```
$ python3 datediff.py  
Enter start date, using format year,month,date: 1970,01,01  
Enter start date, using format year,month,date: 1970,02,14  
The difference is 44
```

```
$ python3 datediff.py  
Enter start date ... : print(100*'haha I am an attacker!\n')  
...  
File "datediff.py",  
...  
ValueError: not enough values to unpack (expected 3, got 1)
```

We can make it better since now we know how to use **split()** and also **int()** for the conversion to an integer.

int() can actually remove zeros from the beginning of the number,

The old error can now be fixed.

The attacker can't use the same trick anymore, but he/she can still get information about the internal code.

Error 3: the input length

```
$ python3 datediff.py
Enter start date, using format year,month,date: 1970,12
Traceback (most recent call last):
  File "/home/python/datediff.py", line 53, in main
    year1,month1,day1 = inputDate("start date")
...
ValueError: not enough values to unpack (expected 3, got 2)
```

```
$ python3 datediff.py
Enter start date, using format year,month,date: 1970,01,01
Enter end date, using format year,month,date: 1970,12,10,9
...
ValueError: too many values to unpack (expected 3)
>>>
```

Even innocent users are affected:

inputDate() requires exactly 3 arguments.

But the error message is not easy to understand. It, moreover, reveals information about the file system.

It is frustrating to get the error about the end date and to need to start again.

Version 3: check the length of the input

```
def inputDate(prompt):  
    date = (input(prompt)).split(',')  
    if len(date) == 3:  
        return int(date[0]),int(date[1]),int(date[2])  
    else:  
        print("invalid date")  
        return inputDate(prompt)
```

We control the length of the input.

If it is 3, OK.

If not, we print a user-friendly message and we call inputDate again.

```
$ python3 datediff.py  
Enter start date ...: 1970,01,01  
Enter end date ...: 1970,12  
invalid date  
Enter end date ...: 1970,12,01,09  
invalid date  
Enter end date ...: 1970,12,01  
The difference is 334
```

Now we have fixed the problem with the number of numbers.

Error 4: input which is not an integer

```
$ python3 datediff.py
Enter start date: 1980,maj,11
Traceback (most recent call last):

  File "datediff.py", line 61, in main
    year1,month1,day1 = inputDate("Enter start date: ")
  File "datediff.py", line 44, in inputDate
    year,month,day = int(date[0]),int(date[1]),int(date[2])
ValueError: invalid literal for int() with base 10: 'maj'
```

Obviously **int()** cannot convert strings which are not digits!

Version 4: check if the input consists of digits

```
def inputDate(prompt):  
    date = (input(prompt)).split(',')  
    if len(date) == 3  
        and date[0].isdigit()  
        and date[1].isdigit()  
        and date[2].isdigit():  
        return int(date[0]),int(date[1]),int(date[2])  
    else:  
        print("three numbers and comma, nothing else")  
        return inputDate(prompt)
```

isdigit() is what we used in Lab 1.

Note: **date[0]**, etc are valid only after we know that:

len(date) == 3.

```
$ python3 datediff.py  
Enter start date: 1963,maj,11  
three numbers and comma, nothing else  
Enter start date: 1963,05,11  
Enter end date: 2020,07,02  
The difference is 20872
```

Now we can recover from input which doesn't consist of three numbers.

Is there something else that can go wrong?

Error 5: Invalid date

```
$ python3 datediff.py
Enter start date ...: 2020,02,30
Enter end date ...: 2021,13,50
The difference is 720
```

Now we handled most of the **formatting mistakes** in the input.

But **semantic mistakes** remain:
the user can write whatever as long
as it consists of three numbers.

We must control that as well?

Version 5: check that input is a valid date

```
def checkDate(year,month,day):
    return ((0 < month <= 12) and
            (0 < day <= monthLength(year,month)))

def inputDate(prompt):
    date = (input(prompt)).split(',')
    if len(date) == 3 and date[0].isdigit()
        and date[1].isdigit() and date[2].isdigit():
        year,month,day =
            int(date[0]),int(date[1]),int(date[2])
        if checkDate(year,month,day):
            return year,month,day
        else:
            print("Invalid date")
            return inputDate(prompt)
    else:
        print("three numbers and comma, nothing else")
        return inputDate(prompt)
```

`0 < month <= 12`

means the usual (in Python!)

`(0 < month) and (month <= 12)`

We already have **monthLength()** in **datediff.py**

Here we have an extra control with **checkDate()**

Call **inputData()** again when there is an error message

Does everything work now?

```
$ python3 datediff.py
Enter start date: 1900,02,29
invalid date
Enter start date: 1900,02,28
Enter end date: -50,3,15
three numbers and comma, nothing else
Enter end date: 50,3,15
The difference is -675683
```

```
$ python3 datediff.py
startdatum: 1980-01-01
three numbers and comma, nothing else
startdatum:
```

OK, we handle the leap years.

We cannot go before BC.
Future work!

We can have an end date before
the start date. OK for this example.

We want to support other formats
too. Future work!

Why not even month names?

What did we do?

We went from

- one "naive" program from Lecture 2, which only
 - focuses on the algorithm
 - expects that nothing goes wrong

to

- a "robust" program, which takes care of
 - the user mistakes
 - adversary attacks.

This is how we should think when we write **production code**, including code which will be used by unknown and potentially adversarial users.

What is left to do?

The program must be more user friendly and accept input in different formats:

- 2020-07-02
- 200702
- 2/7/2020 - this should be read as "7 February" in the USA!
- 2 July 2020

Parsing dates is a broad topic, google for "parsing dates in Python"!

Summary

When processing user-data

1. Validate it before doing anything with it
2. Store it in an appropriate data-structure
 - when the data is in this structure, it is known to be in the right format

A data invariant is a condition which is true for every value of a certain type

- In our example, every date `d` satisfies the condition `checkDate(d)`
- If the program is well organised, you don't have to run this condition everywhere.

Control Structures

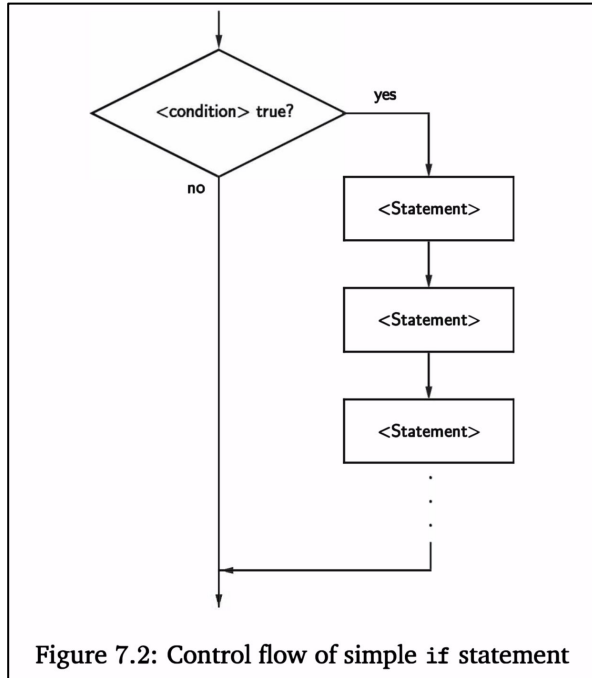
Control Structure

A control structure is any language construct which allows the program to branch in different directions based on different conditions.

Examples:

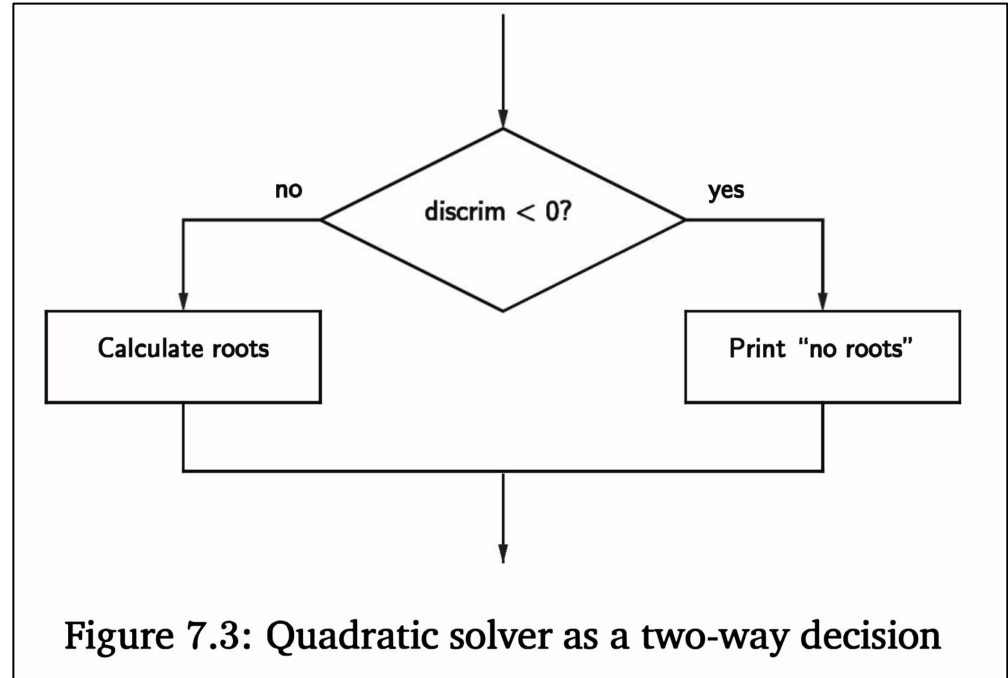
- for-loop
- while-loop
- if-elif-else

Control flow diagram: if-elif-else statements



`if <condition>: <statement>*`

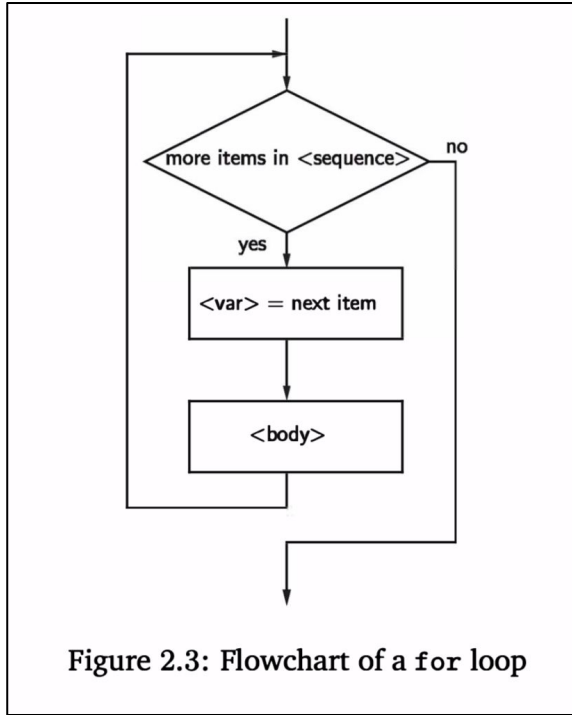
Zelle, p. 213



```
if discrim < 0: print("no roots")
else: <calculate roots>
```

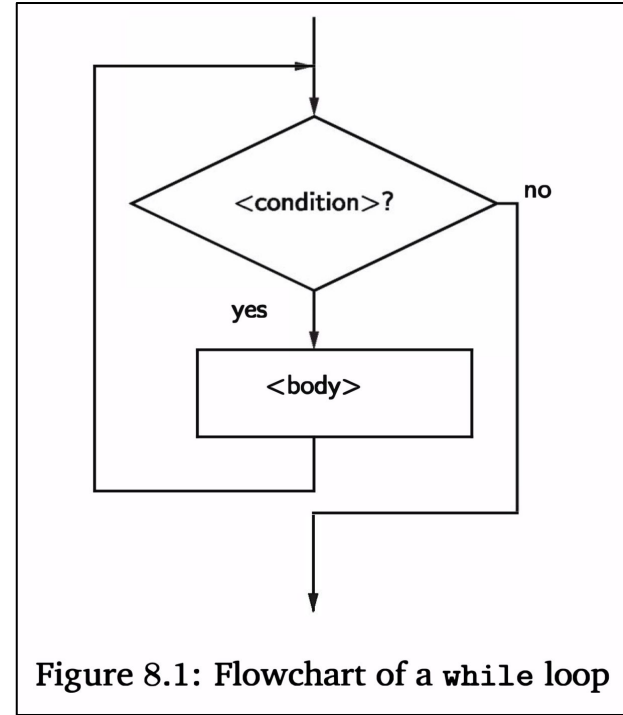
Zelle, p. 219

Control flow diagram: loops



```
for <var> in <sequence>: <body>
```

Zelle, p. 46



```
while <condition>: <body>
```

Zelle, p. 246

Useful idioms for loops and conditions

while-loop over a known sequence: better use for!

```
# salary increment in a number of years
def salaryIncrement(incr,years):
    salary = 1
    year = 1
    while year <= years:
        salary = salary * (1 + incr)
        year = year + 1 # first wrote years!
    return salary
```

```
# could be done with a for loop
def salaryIncrementFor(incr,years):
    salary = 1
    for year in range(1,years+1):
        salary = salary * (1 + incr)
    return salary
```

Implementation based on
while-loops

More natural and shorter with
for-loops

- less code
- clearer logic
- fewer chances of mistakes

While-loop: the number of repetitions is unknown

```
# "repeat until"
def yearsToDouble(incr):
    salary = 1
    year = 0
    while salary < 2:
        salary = salary * (1 + incr)
        year = year + 1
    return year
```

On the previous slide, the number of years is known, which also means that the number of repetitions is also known.

But in this, the number of years is unknown *a priori*

We compute the number of years by incrementing a **counter** in the loop.

(note: The counter year is a simple example for accumulator)

Sentinel-loop: while-loops with sentinel value

```
# sentinel loop
def epidemic1():
    cases = []
    report = ""

    while report != "end":
        report = input("New cases: ")
        newcases = int(report)

        cases.append(newcases)

    print("total", sum(cases))

    if len(cases) >= 7:
        print("7 days average",
              int(sum(cases[-7:])/7 + 0.5))
```

Statistics for an epidemic are collected like this:

An administrator writes the number of new reported cases every day.

But one day they will write that the epidemic is finished.

New cases are saved in a list (to be able to compute other statistics, than just the total sum).

You can report the total cases every day.

But after seven days you can even report the average for the last 7 days (with "Swedish rounding").

Our sentinel-loop was not perfect...

```
$ python3 epidemic.py
new cases: 100
total: 100
new cases: 200
total: 300
...
new cases: 700
total: 2800
7 days average 400
new cases: 200
total: 3000
7 days average 414
new cases: end
Traceback (most recent call last):
  File "epidemic.py", line 46, in <module>
    main()
ValueError: invalid literal for int() with base 10: 'end'
```

Works fine until the epidemic ends

... but after that we see an error.

Post-test loop: validate input before it is evaluated

```
# post-test loop
def epidemic2():
    cases = []
    prompt = "new cases: "
    report = input(prompt)
    while report != "end":
        newcases = int(report)
        cases.append(newcases)
        print("total", sum(cases))
        if len(cases) >= 7:
            print("7 days average",
                  int(sum(cases[-7:])/7 + 0.5))
        report = input(prompt)
```

We test the input-value before it is sent to the loop.

(Maybe we should also test with `isdigit()`)

Disadvantage: the statement which takes input must be written twice.

Now the program ends normally

```
$ python3 epidemic.py
new cases: 100
total 100
new cases: 200
total 300
...
new cases: 700
total 2800
7 days average 400
new cases: 200
total 3000
7 days average 414
new cases: end
$
```

We have not checked that the input is a valid integer - left as exercise!

Loop and half, with a break-statement

```
# Loop and a half
def epidemic3():
    cases = []
    while True:
        report = input("new cases: ")

        if report == "end":
            break

        newcases = int(report)
        cases.append(newcases)
        print("total", sum(cases))
        if len(cases) >= 7:
            print("7 days average",
                  int(sum(cases[-7:])/7 + 0.5))
```

More compact than the previous: no repetition of **input()**

The loop is unlimited because nothing can change the value of **True**!

But we test with the input value, which can lead to terminating the loop—this is what the **break** statement does.

break: terminate a loop early

- **break** terminates the **current loop** before the final condition is met

```
for i in range(-5,6):  
    if i == 0:  
        break  
    print(i)      # this line will run only when i<0
```

```
-5  
-4  
-3  
-2  
-1
```

break and **continue**

- **break** terminates the **current loop** before the final condition is met
- **continue** terminates only the **current iteration** and starts the next one.

```
for i in range(-5,6):    # prints -5 -4 -3 -2 -1 1 2 3 4 5
    if i == 0:           # skips the zero
        continue
    print(i)
```

the same as:

```
for i in range(-5,6):    # prints -5 -4 -3 -2 -1 1 2 3 4 5
    if i != 0:           # skips the zero
        print(i)
```

Searching in Lists

Example: searching for an element in a list

```
def search(list, x):  
    found = False  
    for y in list:  
        if x == y:  
            found = True  
    return found
```

search(list,x) will return True if x is one of the elements in list, and False otherwise.

Can we do it better? The loop always goes over the whole list even if the element has been found.

Example: searching an element in a list

```
def search(list, x):  
    found = False  
    for y in list:  
        if x == y:  
            found = True  
            break          # stop earlier  
    return found
```

Now we don't do more work than necessary.

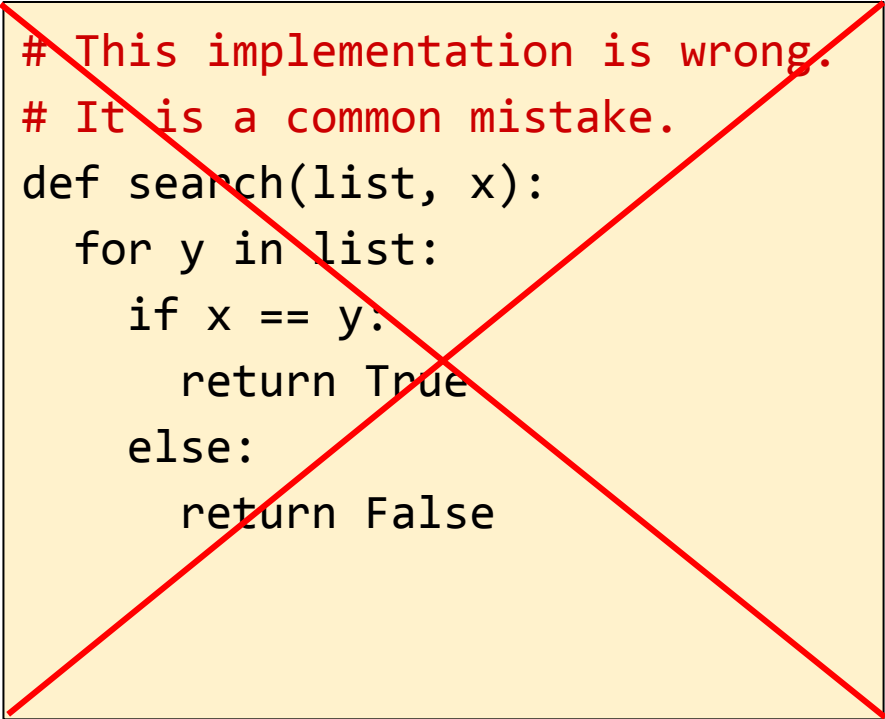
Note: this is a general search pattern, the condition in the if statement can be replaced with something else. For example, search for negative number, primitive number, an interesting movie to watch, etc.

Example: searching an element in a list

```
def search(list, x):  
  
    for y in list:  
        if x == y:  
            return True  # stop the loop  
                           # and return  
    return False
```

If we just want to return True/False, then we don't even need the variable.

Example: searching an element in a list



```
# This implementation is wrong.  
# It is a common mistake.  
def search(list, x):  
    for y in list:  
        if x == y:  
            return True  
        else:  
            return False
```

Instead of searching through the whole list, this function will only look at the first element of the list (if it exists).

This was a common mistake from the previous Java courses.

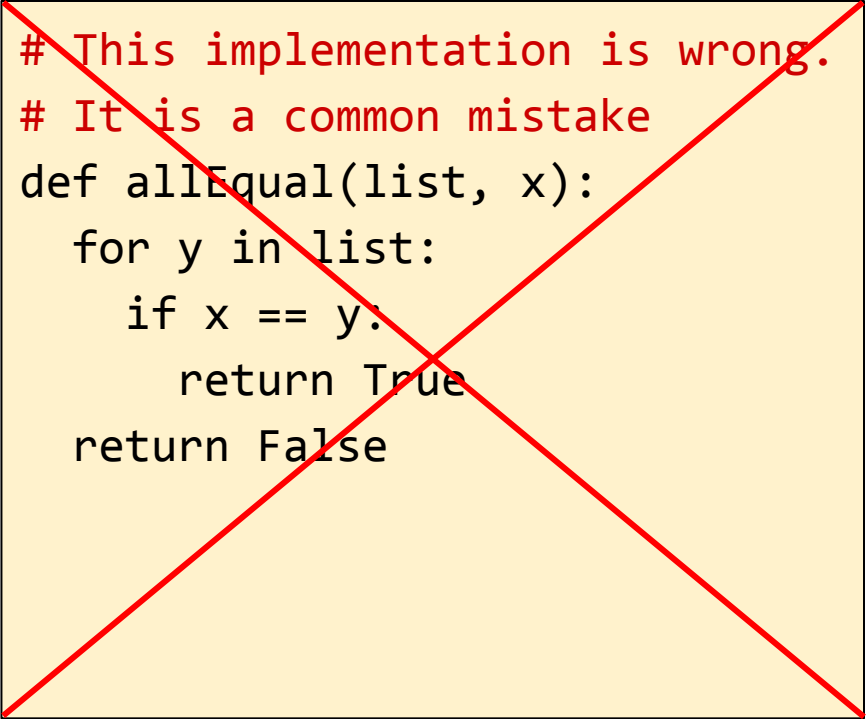
Example: checking all elements

```
# Returns true if all elements  
# are equal to x  
def allEqual(list, x):  
    for y in list:  
        if y != x:  
            return False  
    return True
```

`allEqual(list,x)` returns `True` if all elements are equal to `x`, and `False` otherwise.

Again: this is a common pattern, and the condition can be changed to something else.

Example: checking all elements



```
# This implementation is wrong.  
# It is a common mistake  
def allEqual(list, x):  
    for y in list:  
        if x == y:  
            return True  
    return False
```

`allEqual(list,x)` returns `True` if all elements are equal to `x`, and `False` otherwise.

This implementation will return if the first element is equal to `x`, and it will never check the rest of the list.

List comprehensions

List comprehension (extra material, optional)

```
>>> [x**2 for x in range(10)]  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

```
>>> xs = []  
>>> for x in range(10):  
...     xs.append(x**2)  
...  
>>> xs  
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

List comprehension: an expression for a list build from a sequence.

Similar to the mathematical notation for sets:

$$\{x^2 \mid x \in \{0, \dots, 9\}\}$$

It corresponds to a for-loop but is more compact.

List comprehensions are borrowed from Haskell!

<https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions>

List comprehension with condition

```
>>> [x**2 for x in range(10) if x%2 == 1]  
[1, 9, 25, 49, 81]
```

```
>>> xs = []  
>>> for x in range(10):  
...     if x%2 == 1:  
...         xs.append(x**2)  
...  
>>> xs  
[1, 9, 25, 49, 81]
```

You can choose only elements which satisfy a condition.

If corresponds to an if-statement in the for-loop.

List comprehension for strings

```
>>> [c for c in "Python is fun" if c not in 'aeiouyääå']  
['P', 't', 'h', 'n', ' ', 's', ' ', 'f', 'n']
```

```
>>> def unvocalize(s):  
...     cs = [c for c in s if c not in 'aeiouyääå']  
...     return ''.join(cs)  
...
```

```
>>> unvocalize("Python programming can be fun")  
'Pthn prgrmmng cn b fn'
```

```
>>> unvocalize("nainen ei näe enää")  
'nnn  n n'
```

All characters in the string which are not vowels.

A function which converts that back to string.

This is how, for example, text is usually written in Arabic and Hebrew, i.e. without vowels, and it is still readable.

But works not so well for many other languages (Finnish example: "kvinnan ser inte längre").

Generators

Control flow diagram: loops

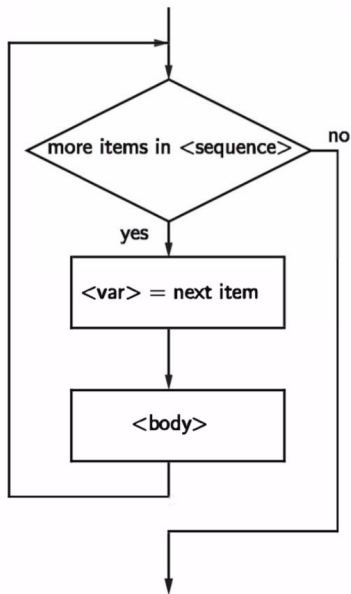


Figure 2.3: Flowchart of a for loop

salary increment in a number of years could be done with a for loop

```
def salaryIncrementFor(incr, years):  
    salary = 1  
    for year in range(1, years+1):  
        salary = salary * (1 + incr)  
    return salary
```

What does “range” really mean here?

for <var> in <sequence>: <body>

Implement your own range as a **generator**

```
def my_range(start, end):  
    i = start  
    while i < end:  
        yield i  
        i = i + 1
```

Use it as the built-in range:

```
for x in my_range(1,10):  
    print(x)
```

Note:

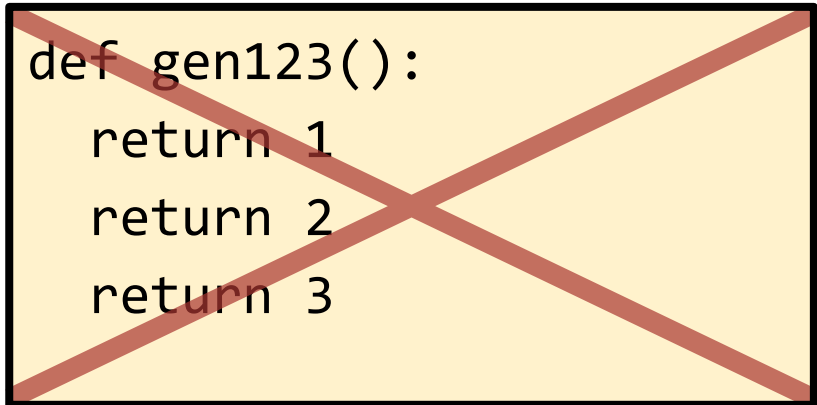
- “return x” - stops the computation of the current function and gives “x” back.
- “yield x” - *suspends* the current computation and gives “x” back. The computation can be resumed later to get another value.

A simpler example

Simply generate the sequence 1,2,3

```
def gen123():  
    yield 1  
    yield 2  
    yield 3
```

This returns only 1!



```
def gen123():  
    return 1  
    return 2  
    return 3
```

We can do this:

```
def gen123():  
    return [1,2,3]
```

but then we have to keep all values in a list. When the list is long we use a lot of memory.

A generator that yields only prime numbers

```
def primes():  
    i = 2  
    ps = []  
    while True:  
        is_prime = True  
        for p in ps:  
            if i % p == 0:  
                is_prime = False  
                break  
        if is_prime:  
            ps.append(i)  
            yield i  
        i = i + 1
```

Use it like this:

```
for p in primes():  
    print(p)
```

WILL RUN FOREVER

Or like this, if you want numbers less than 100:

```
for p in primes():  
    if p > 100:  
        break  
    print(p)
```

Files as generators

```
def mkDict(lines):  
    dict = {}  
    for line in lines:  
        fields = line.split(';')  
        if len(fields) >= 2:  
            dict[fields[0].strip()] = fields[1].strip()  
    return dict  
  
def readDict(filename):  
    file = open(filename)  
    lines = file.readlines()  
    file.close()  
    return mkDict(lines)  
  
def readDict2(filename):  
    file = open(filename)  
    d = mkDict(file)  
    file.close()  
    return d
```

readDict is from Lecture 5.

In readDict2 we pass the file directly to mkDict.

The file is (implicitly) converted to a generator by the for loop. This generator reads the lines from the file, one for each iteration.

The difference is that we can close the file only after we have created the dictionary. This will be more memory efficient, since we don't need to have the whole file in memory.

Exceptions

What is an exception?

An exception is thrown whenever there is a runtime error. Examples:

- Division by zero
- Type errors

External events can also trigger exceptions:

- Control+C pressed
- signal from another process

A program can also explicitly raise an exception

Stack traces

```
def some_function(x):  
    a,b,c = x  
  
def main():  
    some_function([1,2,3,4])
```

```
File "/home/jyp/tmp/x.py", line 5, in main  
    some_function([1,2,3,4])  
File "/home/jyp/tmp/x.py", line 2, in some_func  
a,b,c = x  
ValueError: too many values to unpack (expected 3)
```

By default, when an exception is raised, the python interpreter stops the program, and prints a *stack trace*.

A stack trace contains a list (or *stack*) of function calls and source code locations.

The bottommost entry is the function and location where the error was raised.

The next entry is the caller of the function where the error was raised, and where it was called. The next entry is the caller's caller, etc.

Catching exceptions

Sometimes (typically in interactive programs, servers, etc.) you don't want the program to stop when an exception is raised. Rather, you want to *catch* the exception and recover from the error. This is done using a `try` statement.

Keyboard exceptions: Ctrl+C and Ctrl+D(Ctrl+Z)

```
$ python3 datediff.py
Enter start date: 2020-07-02
three numbers and comma, nothing else
Enter start date: ^DTraceback (most recent call last):
  File "datediff.py", line 92, in <module>
    main()
  File "datediff.py", line 87, in main
    year1,month1,day1 = inputDate("startdatum")
...
EOFError
```

```
$ python3 datediff.py
Enter start date: ^CTraceback (most recent call last):
  File "datediff.py", line 92, in <module>
    main()
...
KeyboardInterrupt
```

Here the user tries to stop the input with Ctrl+D (Ctrl+Z on Windows)

The error message is unreadable and reveals implementation details.

But we see that it is an **EOFError** (end of file)

The same with Ctrl+C

Now we know that it is a **KeyboardInterrupt**

Catching the exception

```
def main():  
    try:  
        year1,month1,day1 = inputDate("start date: ")  
        year2,month2,day2 = inputDate("end date: ")  
        print("The difference is",  
              dateDiff(year1,month1,day1,year2,month2,day2))  
    except:  
        print()  
        print("Ok, see you")
```

```
$ python3 datediff.py  
Enter start date: ^D  
Ok, see you  
  
$ python3 datediff.py  
Enter start date: 2020-04-11  
three numbers and comma, nothing else  
Enter start date: 2020,04,11  
Enter end date: ^C  
Ok, see you
```

try-except looks like **if-else** but we test for Python exceptions.

The **try**-block says what happens if everything goes fine.

The **except**-block says what should happen if something goes wrong.

If an error occurs on any of the lines inside the **try**-block, the rest of the block is skipped, and the **except**-block is executed instead.

Catch the right kind of exception

```
def main():
    try:
        year1, month1, day1 = inputDate("start date")
        year2, month2, day2 = inputDate("end date")
        print("The difference is",
              dateDiff(year1, month1, day1, year2, month2, day2))
    except KeyboardInterrupt:
        print()
        print("if you want to exit, press ^D")
        main()
    except:
        print()
        print("Ok, see you")

if __name__ == '__main__':
    main()
```

Similar to **elif**, we can test for different conditions and do different things.

We do the same as in the Python-shell: ^C doesn't terminate the shell, but ^D does.

Recall this Python-idiom from Lecture 2. Function **main()** is executed only if we start the program from the command line.

finally clauses and with statements

```
def readDict(filename):  
    file = open(filename)  
    try:  
        return mkDict(file)  
    finally:  
        file.close()
```

A variation of the readDict function from earlier.

Sometimes you do not want recover from the exception, but you must execute a piece of code regardless. This is what `finally` clauses are for. This is one of the main uses for exception handlers in production code.

```
def readDict(filename):  
    with open(filename) as file:  
        return mkDict(file)
```

This kind of pattern is so common that there is a special syntax for freeing resources automatically: the `with` statement. Note that python automatically knows what method is the finalizer to call. (By using a special convention for class methods, it knows that `close` is the finalizer for files.)

References

From the book

- Chapters 7 and 8 except 8.6 (which is about graphics)
- Generators
- List Comprehension <https://docs.python.org/3/tutorial/datastructures.html#list-comprehensions> (optional)